

AFRILANG Stdlib

std/c/graphcycle

Bibliothèque AFRILANG `std/c/graphcycle` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
detectCycleDir

```
`import "std/c/graphcycle" · `use graphcycle`
```

- `detectCycleDirected(adj list number, n number) ? bool`
- `detectCycleUndirected(adj list number, n number) ? bool`
- `cycleEdgeCount(adj list number, n number) ? number`
- `isAcyclic(adj list number, n number) ? bool`
- `selfLoopCount(adj list number, n number) ? number`
- `removeSelfLoops(adj list number, n number) ? list number`
- `hasParallelEdges(adj list number, n number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphcycle` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : detectCycleDir

```
`import "std/c/graphcycle" . `use graphcycle`  
- `detectCycleDirected(adj list number, n number) ? bool`  
- `detectCycleUndirected(adj list number, n number) ? bool`  
- `cycleEdgeCount(adj list number, n number) ? number`  
- `isAcyclic(adj list number, n number) ? bool`  
- `selfLoopCount(adj list number, n number) ? number`  
- `removeSelfLoops(adj list number, n number) ? list number`  
- `hasParallelEdges(adj list number, n number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphcycle"  
use graphcycle
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? detectCycleDirected(adj list number, n number) ? bool  
? detectCycleUndirected(adj list number, n number) ? bool  
? cycleEdgeCount(adj list number, n number) ? number  
? isAcyclic(adj list number, n number) ? bool  
? selfLoopCount(adj list number, n number) ? number  
? removeSelfLoops(adj list number, n number) ? list  
? hasParallelEdges(adj list number, n number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphcycle"  
use graphcycle  
  
create result = detectCycleDirected(/* args */)   
say result  
  
create result = detectCycleUndirected(/* args */)   
say result  
  
create result = cycleEdgeCount(/* args */)   
say result  
  
create result = isAcyclic(/* args */)   
say result  
  
create result = selfLoopCount(/* args */)   
say result  
  
create result = removeSelfLoops(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/graphcycle"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module graphcycle

```
function detectCycleDirected(adj list number, n number) returns bool  
end
```

```
function detectCycleUndirected(adj list number, n number) returns bool  
end
```

```
function cycleEdgeCount(adj list number, n number) returns number  
end
```

```
function isAcyclic(adj list number, n number) returns bool  
end
```

```
function selfLoopCount(adj list number, n number) returns number  
end
```

```
function removeSelfLoops(adj list number, n number) returns list number  
end
```

```
function hasParallelEdges(adj list number, n number) returns bool  
end  
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphcycle` (tier complex, category complex). Exports 7 function(s): detectCycleDirected, detect

```
`import "std/c/graphcycle" . `use graphcycle`  
- `detectCycleDirected(adj list number, n number) ? bool`  
- `detectCycleUndirected(adj list number, n number) ? bool`  
- `cycleEdgeCount(adj list number, n number) ? number`  
- `isAcyclic(adj list number, n number) ? bool`  
- `selfLoopCount(adj list number, n number) ? number`  
- `removeSelfLoops(adj list number, n number) ? list number`  
- `hasParallelEdges(adj list number, n number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphcycle"  
use graphcycle
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? detectCycleDirected(adj list number, n number) ? bool  
? detectCycleUndirected(adj list number, n number) ? bool  
? cycleEdgeCount(adj list number, n number) ? number  
? isAcyclic(adj list number, n number) ? bool  
? selfLoopCount(adj list number, n number) ? number  
? removeSelfLoops(adj list number, n number) ? list  
? hasParallelEdges(adj list number, n number) ? bool
```

4. Usage examples

```
import "std/c/graphcycle"  
use graphcycle  
  
create result = detectCycleDirected(/* args */)   
say result  
  
create result = detectCycleUndirected(/* args */)   
say result  
  
create result = cycleEdgeCount(/* args */)   
say result  
  
create result = isAcyclic(/* args */)   
say result  
  
create result = selfLoopCount(/* args */)   
say result  
  
create result = removeSelfLoops(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphcycle"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphcycle
```

```
function detectCycleDirected(adj list number, n number) returns bool  
end
```

```
function detectCycleUndirected(adj list number, n number) returns bool  
end
```

```
function cycleEdgeCount(adj list number, n number) returns number  
end
```

```
function isAcyclic(adj list number, n number) returns bool  
end
```

```
function selfLoopCount(adj list number, n number) returns number  
end
```

```
function removeSelfLoops(adj list number, n number) returns list number  
end
```

```
function hasParallelEdges(adj list number, n number) returns bool  
end  
end
```